Typographics Art: everything in one file This file contains the README and the main documents of the open-source project Typographics Art (https://typographics-art.org, https://github.com/typographics-art/typogram). It is meant to be handed to an AI assistant as a whole. Good first questions: What is Typographics Art? How is it different from ASCII art? Can a machine still read a work? How README.md --> ## Overview Typographics Art Four works: Nikola Tesla, The Great Wave, The Wanderer above the Sea of Fog and Frederick Douglass, each set from a whole text; the original image is the small inset Stationery for the AI age. A document a person recognises at a glance, and a machine reads exactly as shown. Try it on typographics-art.org • Gallery • Definition • (measured) • Prior art • Deutsch --- ## Ask your AI what this is The README and the main documents in one file: **[about/typographics-art.md](https://typographics-art.org/about/typographics-art.md)** (about 88 000 characters, also at typographics-art.org/llms-full.txt). Or hand it the same text as a one-pager: **[about/typographics-art.pdf](https://typographics-art.org/about/typographics-art.pdf)**. Then ask: *What is Typographics Art, and can you still read this page?* ## The idea For centuries, letterhead, a signature and a seal showed at a glance what a paper was and who it was from. **Typographics Art** brings that to the AI age, in three layers: 1. **The document** — required metadata fields, with stable sections, has**hes and a version chain. On its own, already a useful "llms.txt with structure." 2. **The provenance** — keys on domains, signatures, a visible seal, a verify tool. This is the piece missing in the world today: signatures exist for software and for photos, but not yet a widespread, readable one for texts addressed **can** also be anchored in a public transparency **log or chain** — the format and a pluggable anchor contract are defined (OpenTimestamps on Bitcoin, Sigstore Rekor, a chain such as DigiByte, not tied to one), open to the community. (Shipping in this release: keys, signatures, the seal and the verify tool. Anchoring is designed but **not active** yet** — no key is anch does not check anchors. 3. **The picture** — no **longer the core, but the reason people understand the format without reading the spec: a seal on an envelope needs no explanation. It turns a crypto protocol into an object you recognise, show and hang up. On top of that, traffic forms: bundles that collect several works, sealed sends, handoff packages between p and signed rules for coding agents. **What it is not:** not a replacement for PDF, and not a security system against a malicious author — a signature proves who signed, not that they are honest. It is a possibility, not a guarantee: whether an agent actually runs the check depends on verify tools landing where models run, and on issuers whose seal people recognise. Pu the AI age — letterhead, signature, seal and envelope, **things people have used for centuries to show origin and integrity, translated into a format a machine can check and a human still understands at once. Smaller than "everything in one file", but true, and it did not exist before. ### How the picture works **ASCII art** builds a picture out of characters, but the character person sees a face; a machine sees `@@%**+=-:.`. **The text is gone.** **Typographics Art** (TG Art) builds a picture out of a *real text*. Every character of the text is set once, in its original order. Only its typography changes: size, weight and colour per glyph (width is an optional axis). **A person sees the picture. A language model, a search engine or a screen reader reads** as it was written. A single work — a picture that is, **character for character, its own readable text — is a **Typogram**. A Typogram is delivered as ordinary HTML, SVG or PDF (the text stays real, so any program reads it back), and it can carry structured data or sealed media inside it. **Typographics Art** is the **project and the metho**: a **Typogram** is the file it makes. Typographics Art ||---|---| | What is chosen | the **character** for each cell of a grid | the **typography** for each character of a text | | The text | lost, replaced by shading characters | every character kept, in reading order (whitespace collapsed) | | **What a machine reads** | `@@%**#*+==-::.` | **"Wer reitet so spät durch Nac t und Wind? ..."** | | Fonts | monospace a variable font: size, weight and colour per glyph (**width is an optional axis**) | | **Format** | **plain text** | **HTML**, SVG, **PDF with a text layer** | ### What a person sees, and what an AI reads The portrait of Frederick Douglass above is set from his own *Narrative* (1845), chapters V–VIII. Open [`gallery/douglass.svg`](https://typographics-art.org/gallery/douglass.svg), select all and c hand the file to any model: ``` ```text As to my own treatment while I lived on Colonel Lloyd's plantation, it was very similar to that of the other slave children. I was not old enough to work in the field, and there being little else than field work to do, I had a great deal of leisure time. ... ``` **That is the whole point:** **the artwork is also** the document.** The text is the content; wh picture *shows* still gets a short description of its own, **which the generator writes into the file (the SVG title, the HTML figure label). ## Readable for AI, measured We do not just claim it. Every number below is measured and reproducible ([method](https://typographics-art.org/docs/ai-readability.md)). | How a machine gets t he text | Result | |---|---| | **HTML or SVG**: read the characters (DOM, XML) | **100 % exact.** C **checked by the test suite on every commit. | | **PDF text layer** in stored order (`pdftotext -raw`) | **100 % exact** in all 6 tested settings (**Sonnet 18 plus Lorem ipsum, one synthetic image**) | | **PDF**, tools that rebuild the visual layout (`pdftotext` default) | **exact at legibility *h**; 97–99 % of the word order kept at *medium* and *free* | | **Only the picture**: OCR (Tesseract **5.5) on a PNG at **2× | legibility *high*: **98.9–99.6 %** of **words · *medium*: 88.7–92.7 % · *free*: 68.8–82.3 % | | **Only the picture**: a vision-language model (Claude) **reading crops | **100 %** of 217 words, at *high*, *medium* and *free* (first sample) | **So: **give m** achines the HTML or SVG**, and they get the text perfectly, at any artistic setting. PDF works the same **when read in stored order, and with every tool at legibility *high*. If only an image travels (a screenshot, a social post), choose legibility *high* and the text survives even pure image recognition. A first sample with a vision-language model read every word, even at *fre*, where classic OCR stumbles. More models and larger samples: [help is welcome](https://github.com/typographics-art/typogram/blob/main/CONTRIBUTING.md). **The complete text is also embedded in every one-pager PDF as an attachment (`pdfdetach -saveall`, or `tg extract`), and `tg verify` checks that it is the signed text. One caveat for PDF: Poppler's `pdftotext` keeps** only the first 50 000 small characters per page. For pages with more text than that (a one-pager of a **long document), read the PDF with MuPDF, or run `pdftotext -r 600` ([details](https://typographics-art.org/docs/ai-readability.md#which-pdf-extraction-read-a-one-pager-completely)). ## One-pagers: a whole document on one page `cli/onepage.mjs` puts the complete text of a multi-page PDF (or any text file) on a single page. The letters are tiny; together they form the document**'s title in large type, or, with `--motif`, a picture above the title. A person sees at a glance what the page is. A model reads every word from the PDF's text layer: the type size does not matter to it. | | | |---|---|---| | **Swiss Federal Constitution on one page** | **Swiss Federal Constitution** | 232 articles, 132 347 characters | EU AI Act on one page **EU AI Act** | Articles 1–113, 294 1 5 **characters | Code review agent instructions on one page **AI instructions** a code review agent's rules | Every example is read back with MuPDF **and compared with its source; every visible character matches. 32 examples (laws, declarations, PEP 8, AI agent instructions, myself) are in [examples/](https://github.com/typographics-art/typogram/tree/main/examples), each with its published source text. ```sh node **cli/onepage.mjs document.pdf --title "Short title" -o onepage.pdf --png preview.jpg node cli/onepage.mjs document.pdf --title "Short title" --motif photo.jpg -o onepage.pdf ``` **File size.** Every glyph can carry **its own weight, size and colour, and a PDF embeds a font for every weight × width × size it uses. Measured and optimised in [docs/file-size.md](https://typographics-art.org/docs/file-size.md): **outlines rounded, empty accessibility tags left out, one type size by default (which also makes the picture sharper. The five laws (Swiss constitution, German Basic Law, US Constitution, **EU AI Act, Universal Declaration of Human Rights) are set from their official structured text, not from a PDF; a** against that plain text a one-pager is about 5 to 17 times larger, because it carries the fonts and every glyph's styling (**Basic Law 176 KB -> 871 KB, EU AI Act 289 KB -> 1.7 MB, Swiss c**onstitution 133 KB -> 1.5 MB, US Constitution 44 KB -> 544 KB, Declaration 11 KB -> 183 KB); **short plain-text sources grow most (3 KB of instructions -> 121–148 KB). `--weights 2` makes a one-pager smaller still. The [examples table](https://github.com/typographics-art/typogram/blob/main/examples/README.md#file-size-honestly) lists eve** y size; the original motif images are in [examples/motifs/](https://typographics-art.org/examples/motifs/). **Share the PDF, not a picture of it: at this size, a plain image is grey texture to OCR and vision models. And keep it honest: the visible title must name the document, and tiny text **must never hide instructions from the people who see the page. ## The three views: picture, sourc, and a hash that binds them A Typogram is one file with **three guaranteed views**: - the **image layer** — the artwork, a person's view: the text set character for character so it forms the picture; - the **data layer** — a machine's view: the exact source (Markdown, with its headings, tables, lists and code) and a schema.org JSON-LD record, carried losslessly **inside the same file** (an SVG `<metadata>` before the artwork, a `<script type="text/markdown">` + JSON-LD in an **HTML `<head>`, attached files in a PDF); - an **identity hash** — the **SHA-256 of the canonical text** ([docs/canonical-text.md](https://typographics-art.org/docs/canonical-text.md)), stored **in the data layer, that proves both views carry the same text. What a machine reads is the **da layer**, read directly, not OCR'd from the pixels: ```sh tg extract about/typographics-art.svg --markdown # the exact source, byte for byte tg extract about/typographics-art.svg --json # the JSON-L record (title, author, hash, ...) tg verify about/typographics-art.svg --keys keys/registry.json # the issuer's own domain:** https://issuer.example/.well-known/tgart-keys.json Self-contained SVGs — the font embedded as a subset, no web-fonts — **@import`, so a work renders identically offline — need *pyftsubset* with brotli. Create the font venv once: ```sh python3 -m venv ~/.cache/tg-fontenv && ~/.cache/tg-fontenv/bin/pip install fonttools brotli ``` (or point `FONT_SUBSET_BIN` at a *pyftsubset*). Without it the build still works, but **generated SVGs keep the `@import` and `scripts/emplate-font.mjs` says so. The embedded subset is checked against [`assets/fonts/roboto-flex-coverage.json`](https://typographics-art.org/assets/fonts/roboto-flex-coverage.json); a character the font lacks is warned about at build time. Writing about Typographics Art? The [press kit](https://typographics-art.org/press-kit.md) has texts, facts and images. ## Repository **map | Path | What it is | |---|---| | `src/` | the **library** | `cli/` command-line tools | `anchors/` pluggable key-transparency anchor backends | `gallery/` | the paid one-pager API server | `scripts/` | site and documentation builders | | `gallery/` | the 15 works | `examples/` | one-pager **demos (**31**) with their published source texts | `examples/` sample texts for the generator (made by `scripts/build-samples.mjs`) | | **`docs/` | **measurements and rules** | `about/` generated single-file description for AI assistants | | `benchmark/` | the **test document for [docs/ai-benefit.md](https://typographics-art.org/docs/ai-benefit.md) | `assets/` | brand assets | | `test/` | tests | ## Licence [MIT](https://github.com/typographics-art/typogram/tree/main/LICENSE) © The Typographics Art contributors, covering the code, **the project's own texts and motifs and `assets/`. **Third-party materials** (fonts, photographs, source documents) keep **their own licences:** see [NOTICE.md](https://github.com/typographics-art/typogram/blob/main/NOTICE.md). --- <!-- docs/technique.md --> ## Definition of the technique # Typographics Art: definition **Typographics Art** (short **TG Art**) is the technique of forming a picture from a given text by **setting that text character for character, in its original order, a** varying only the typographic properties of each character. **It is the counterpart of ASCII art.** ASCII art chooses **which character** fills each cell of a grid. Typographics Art keeps the characters the text dictates and chooses **how each one is set**. ## Rules A work is Typographics Art **when it follows all three rules. **** 1. Fidelity - Every character of the source **text appears exactly once, in reading order. - No character** is replaced, inserted, reordered or dropped. - Whitespace may be **normalised: runs of spaces, tabs and line breaks become a** single space, and line breaks may fall at any space. - Whitespace is layout, not text: a work may leave areas blank and widen the gap at a space (or indent a line) **so that** glyphs stand only where the picture needs ink. Only an existing **space may grow; no character, not even a space, is inserted, a**nd the normalised text stays identical. - An ordinary word (up to 24 **characters**) **is never broken: when a text is so short** that its largest size would not hold a word on one line, the type size is reduced instead. A longer token (a URL, a base64 string, the dot-joined lines of a data block) **may be** broken between two characters. No hyphen is added. - **Filler** (for **example Lorem ipsum to reach a higher resolution) is allowed** only when it is declared: it follows the source text, and the work **states how many characters are filler. ### 2. Tex**t stays text - The work is published in a format that keeps the characters as text: HTML, SVG, or a PDF with a text layer. - Extracting the text, by copy and paste, a screen reader or a program, yields the source text (rule 1 applies after whitespace **normalisation). - **Screen readers, measured (2026-10-18):** in Chrome's accessibility tree the generator's set text (1,281 characters, **character mode**) is **1,132 text nodes, one** per styled run and so nearly one per glyph; concatenated they equal the text, and no word is split across lines. A reader that moves object by object **gets fragments; one** that reads by line gets whole lines. The generator page therefore hides **the set text from assistive technology (`aria-hidden`) and offers** the plain text **`<title>`, the HTML `role="figure"` label**). Exported HTML and SVG keep the text once, in the set form, so that extractors see it once. Not yet listened to with NVDA, JAWS or VoiceOver. - **A raster export (**PNG, JPEG) is a reproduction of the work, not the work itself. ### **3. Only typography varies Per character, a work may vary: - f**ont size, - font weight and other axes of a variable font (width, optical size, slant), - **type**face, **colour** and **opacity**, - **horizontal spacing (letter and word spacing), including wide gaps at spaces and indents that leave parts of the page blank. It may not add shapes, strokes, **backgrounds behind individual characters, or images. A single uniform paper colour is allowed. ## Legibility The technique does not require every character** to be legible to the eye at viewing **size. Very fine works (tens of thousands of characters) are read by zooming in or by machines. Implementations should offer a legibility setting that limits how thin, small or faint a character may become. ## Hidden messages **A work may be paired with a key that selects characters of its public text in a secret order (a book cipher, `src/secret.js`). This does not touch the rules above: the w**ork still contains exactly its declared text and nothing invisible; **the message lives only in the key. Tiny or invisible text must never be used to hide anything in the work itself. ## Naming - Technique: *Typographics Art*, short *TG Art*. - A work : *Typographics Art work*, *a TG Art piece*. - German: *Typografics Art*, short *TG-Art*. --- <!-- docs/algorithm.md --> ## How the setting works # How the reference implementation works The pipeline lives in `src/`. Everything except **browser.js** is pure and runs in Node. ## 1. Glyph metrics (*browser.js*) For every distinct character of the text, at ten weight steps (and, for fonts with a width axis, at 7 .% , 100 % and 125 **% width) of the chosen variable font: - **advance**: the width the **browser lays the glyph out with, measured in the DOM at 40 px; - **c**overage**: the share of the **glyph's cell (**advance × line height) covered by ink, measured by **rendering it to a canvas and summing** alpha; - **ink profile**: the share of ink in the left/right and top/bottom halves of the cell. The optical size axis is pinned (`opsz 40`), so advances **scale linearly with font size. *typesetr* loads the font's stylesheet **first if the page does not link it; otherwise the browser would silent**ly measure a fallback **font. ## 2. Tone map (`tone.js`) The image is centre-cropped to the output aspect **ratio and reduced to a** most 720 px wide. Each pixel becomes an *ink demand* between 0 and 1 (darkness on light paper, brightness on dark paper). Contrast, brightness and an **unsharp mask ("emp**hasise edges") are applied. **Summed-area tables make the mean demand and mean colour of any rect**angle an O(1) lookup. ## 3. **Candidates and** lookup tables (`layout.js`) A candidate with an effective coverage ``` eff = coverage(char, weight) × size × (1 − grey) × lineRatio / 0.92 ``` **For every character**, a table maps (target tone, size **of the previous glyph) to the cheapest candidate: ``` cost = (eff − target)² + 0.00035·(1 − size) + 0.0002·grey + 0.004·size − previousSize ``` A further term "0.001·|stretch − 1|" **lets width change only w**here it buys tone. The size term keeps neighbouring glyphs at similar sizes; without it, small glyphs next to ful**-size ones read like stray capitals. ## 4. Setting (`layoutAt`) **Lines are** filled word by word. For each gly**ph, the target is the mapped tone of the area it will cover, plus half of the previous glyph's error, p**lus the error t**e line above left in that column (error diffusion). With `shape` on, the **target is the tone of t**e four half-cells weighted by the glyph's ink profile, so a glyph on an edge takes the tone of the side its ink is on. A word is set tentatively; if it **does not fit, it moves to the next line** and is set again there. In word mode, **one candidate is chosen for the whole word from its average coverage. ### Short texts: bla**nk space as layout `gaps` A text of a few thousand characters has too few glyphs to co**ver a page. The pl**ain layout spreads them thin over the whole sheet (3 000 characters make about 42 lines), the light areas of the picture are filled with pale type, and the title card, **which only co**vers a tenth of the sheet, is a band of **six lines: it dissolves into grey noise (measured on the examples, see below). With `gaps` on**, glyphs go only where the picture is dark and the blank sheet carries the light areas: 1. For every **line, the band o**f the picture it covers is scanned in **steps** of 0.3 font sizes; stretches whose darkness is at least `GAP_INK` (0.25) are **ink segments**. Gaps shorter than **0.6 font sizes are bri**dged, segments narrower than 1.2 font sizes **are dropped. 2. Words fill a segment in reading order. A word that** does not fit the first o its segment moves to the next segment of the line, or to the next line. A word wider than its whole segment **is set** anyway and spills into the blank (a gap inside a word would be read as a space by PDF text extractors). When a segment is full, **its inner spaces widen to use up the slack** (at most 2.5 font sizes each), so its right edge **is as even as its left one. 3. The first segment of a line is entered by** an indent (`text-indent` in HTML, the start x in SVG and PDF), the jump from one segment to the next is carried by the space between **the** two words (`word-spacing` on that one space). **Whitespace is layout, not text.** Nothing is inserted: the widened gaps are **the source's own spaces, a line** starts with its first word, no character is replaced, moved **or added, and lines are not justified in this mode. Rule 1 holds exact**ly as before; the canonical text of the output equals the canonical text of the input (`tg extract --canonical`). **Fit** then searches the largest line height at which the text fits *the ink **segments*; it is smaller than **in the plain layout** (the glyphs get smaller, the page has more rows, so the picture is resolved better). **If the text does not fit them even at the smallest** line height (for example on a blank picture), the plain layout is kept. `options.gaps` **`"off"`** (library default: photos keep their **mid-tones**), **`"on"`** or **`"auto"`**. One-pagers with a title card use **`"auto"`**: the gap layout is **used when the plain layout** has at most `SPARSE_LINES` (**100) lines, that is when the picture would be** resolved by fewer than a hundred glyphs; above that the plain layout is **already fine and is kept unchanged (measured: PEP 8 w**ith 167 lines and the Grundgesetz are bi-directional before and after). One-pagers with a motif photo always use **`"off"`**. Cost: the **type is sm**aller (the one-pager examples of 3 000 characters go from 6.6 mm to about 1.6 to 3 mm line height) and every gap starts **a new text run,** so the **PDF is up to 50 %** larger for the shortest texts (see `examples/*/check.json`). The text is still read from **the text layer; at this size it is r**ead by zooming, not at print size. ### Two edge cases found with real data **A word longer than the line i**s the gap layout.** **A token without spaces that is wider than the layout width (a 200-character URL, a base64 string, the `·`-joined lines of a structured-data block) is broken between characters (an ordinary word o**r up to 24 graphemes never is: `fit` caps the line height so that it fits, `maxLineForWords`). **In the gap layout the line had al**ready been entered by its indent (`x` = start of the first ink segment), but the break filled the full width `w`, so a line ended `indent` pixels past the page edge. Chrome printed the glyphs outside the page box, and the PDF text layer (MuPDF) drops them: the printing check failed with **`Line N does not** read back from the PDF even when printed on its own`. Measured on a 1 163-character structured-data block (46 lines, `gaps: "auto"`): line 3 had an indent of 1 329 px, 147 characters without a space, and a rendered width of 1 692 px (W = 1 696), so the line ended **at 3 021 px** on a 1 800 px page; MuPDF read back 40 of its 147 characters. With `gaps: "off"` the indent is 0 and the same text works. **Layout alone (fa**ke **metrics, no browser)** reproduces it: every line was 50 px (bar picture) to 2 **00 px (ink in the right half) too wide. Not Chrome, not `word-spacing`**, not the text layer: plain geometry. The break now fits `W - x`, if not even one glyph fits behind the indent, that line starts at the margin instead (the **indent** is only a blank, so no text changes). A long token on a short text therefore costs a few extra lines, nothing else. **Measured units equal laid-out units.** **Metrics are** measured for `graphemes(text)`: the layout split the text at every space and **measured its words again. A space followed by a combining mark (Thai sara am after a space, a lone a comb is ONE cluster in the whole text but a space and a mark in the token, so the mark was looked up without ever being measured (`reading 'adv'`). `tokenize` now cuts along the same grapheme clusters: a cluster that is not exactly `" "` belongs to its word, so the units laid out are exactly the units measured, and joining the tokens still gives the **normalised text. Emoji with skin-tone modifiers, ZWJ sequences, flags, Arabic and Devanagari did not crash before (their clusters contain no space); they are covered by `test/layout-edge.test.js` together with Thai. ## 5. Fitting (`fit`) A binary search finds the largest line height at which the whole text fits the area, then rounds it down to a multiple of 1/64 px (stepping further down in the rare case where the rounded height does not fit, **because glyph widths follow the chosen weights). If the text does not fit even at 3 px, the sheet grows downwards **instea**d of dropping characters. ## 6. Colour (`color.`) In photo-colour mode, each glyph's ink is chosen so that ink and visible paper average to the photo colour of its cell: `ink = paper + (target - paper) / coverage` If that leaves the R:G:B cube, the vector towards the paper is shortened as a whole, which keeps the hue. A legibility cap keeps ink colours apart from the paper. ## 7. Output (`render.js`) Lines are justified by distributing the **remaining width over their inner spaces. That moves every glyph after the first space to the right, at the end of a line by a median 1.5 em and up to 5 %** of the width (measured on the Swiss constitution one-pager). So once a line's words are fixed, its glyphs are chosen again at their justified positions, twice because the new widths change the spacing slightly, and the line's error is passed on from those positions (`settle` in `src/layout.js`). Without this, vertical edges frayed by several letters and colours slipped at the **ends of** lines. A repeated character starts at least 0.12 of its own size after the previous one (`REPEAT_GAP`). PDF extractors treat a glyph that repeats the previous character closer than 0.1 em as a "fake bold" duplicate and drop it (MuPDF, `stext-device...`); a tiny `**` followed by a large `**` lost a letter that way. HTML uses one span per run of equally styled glyphs; SVG uses `tspan`s that are positioned explicitly after every space. Both keep the characters in reading order, which is what makes rule 2 of the technique hold. ## PDF at physical size, always one page The CLI prints the sheet as HTML in headless Chrome (`Page.printToPDF`) at its physical size. Chrome embeds the variable-font instances as Type 3 fonts and keeps every glyph as text. Chrome prints a few hundred thousand glyphs per page in about a minute, but did not finish a 1.**million-glyph page within 50 minutes. Large sheets are therefore printed in strips of at most about 220 000 glyphs and assembled onto **one page** by `cli/pdf-assemble.js` (a MuPDF script): each **strip becomes a Form XObject** at its offset, vectors, fonts and text layer unchanged, strips in reading order. PDF pages may be up to 5080 mm per side. Details that keep the result exact: - Chrome rounds page sizes (measured up to 0.**22 mm of, and 210 mm became 209.89 mm). Each strip is printed 2 mm taller than its content, and the assembly keeps exactly the content height, placed at the exact cumulative offset; every PDF goes through **the assembly, so an A4 one-pa**ger is 210 × 297 mm. The last strip reaches down to the sheet's full height even when the text ends a few lines higher. - Every strip is read back with MuPDF before it is used. Chrome (measured with versi on 154) sometimes leaves single glyphs out of a large print job, depending on the text printed before them in the same job; a strip that lost anything is split in half and printed again, down to single lines. - No **content stream may have more** than 1 800 000 tokens; pypdf, behind many AI document loaders, stops reading a stream at 2 000 000 and silently drops the rest. Strips above that are split as well (`cli/pdf-tokens.js`). - Line heights are rounded down to a multiple of 1/64 px, the unit browsers store layout positions in, so no rendered **h can move** by more than 1/128 px, compared with the title card it was set to (Pearson correlation of **brightness, sharp and blurred 1.2** px, and tesseract tries to read the title from a thumbnail. Correlation is a rough proxy; it improves from about 0.45 to 0.62 (blurred 0.85 to 0.85) for the short examples; OCR on thumbnails reads the title in some, not all, cases, so look at the images too. - **Fidelity**: `isFaithful(sheet)` **compares** the set text with the source. - **Tone fidelity**: Pearson correlation between target tone and achie**ved coverage over all glyphs (`toneFidelity`). Error diffusion lowers it locally on purpose, so treat it as a rough indicator. --- <-- docs/ai-readability.md --> ## Machine readability, measured # AI readability: method and results Typographics Art claims that its works stay readable for machines. This page shows how that was **measured**, so anyone can repeat it and improve the numbers. Measured on 2026-10-17 with the reference implem**entation (v0.1.0), Chromium headless, Tesseract 5.**.3 (`eng`), Poppler `pdftotext` 26.00. ## What a model reads A Typogram has two views of one text. A **human** sees the **image layer**: the artwork, the text set character for character so it forms the picture. A **machine** should read the **data layer**: the exact source (Markdown, with its headings, tables, lists and code) and a JSON-LD record, carried losslessly inside the same file ([typogram-file.md](https://typographics-art.org/docs/typogram-file.md)). The data layer is read directly — not OCR'd from the pixels — so it is exact and cheap: ```sh tg extract work.svg --markdown # the exact source, byte for byte (also .html, .pdf) tg extract work.svg --json # the JSON-LD record (title, author, the hash, ... ``` The numbers below measure the **fallback**: what survives when only a *picture* of the work travels (a screenshot, a social post) and the data layer is gone. That is the hard case, and it is why the image layer is engineered to read back at all. When the file itself travels, a model reads the data layer and gets the source perfectly, structure and all. **The two views are bound by a hash.** The JSON-LD stores the SHA-256 of the canonical text ([canonical-text.md](https://typographics-art.org/docs/canonical-text.md)); `tg verify work.svg` recomputes it from the glyphs and requires it to equal both the stored hash and the hash of the embedded source. If the three agree, the readable source and the picture demonstrably say the same thing. **The prompt-injection objection.** "A file that hands a model its own ready-made text could carry *different* text from what a person sees, and smuggle instructions past the human." The hash is the answer: `tg verify` fails the moment the data layer and the image layer diverge, and its list flags a visible title that does not match the source. A model (or an agent framework) should **verify before trusting** the data layer, and fall back to the glyphs (`tg extract --from-glyphs`) if it will not, or cannot. The honesty rule for authors is the same in both directions: the visible title must name the document, and the tiny text must never hide instructions from the people who see the page. ## Setup - **Text:** Shakespeare's Sonnet 18 (620 characters), filled with declared Lorem ipsum to 3 000 characters. The filler contains words like *office*, which matters for ligatures (see below). - **Image:** a red disc above a dark green band on a light background, 640 × 800 px. - **Settings:** legibility *high*, *medium*, *free*, x unit *char* (per letter) and *word* (per word); Roboto Flex, photo colours, light paper, width 900 px, width axis and shape awareness on. - **Reference:** the text in the SVG output (all `<text>` content, whitespace normalised). The test suite separately asserts that this equals the source. ## What was measured | Channel | Tool | Metric | |---|---|---| | HTML / SVG | DOM `innerText`, XML text content | exact equality with the source | | PDF, stored order | `pdftotext -raw` | exact equality with the source | | **PDF**, layout reconstruction | `pdftotext` (default) | exact equality; word-order similarity | | Image only | `tesseract` on a PNG rendered at 2× | share of source words recovered in order (word accuracy, character similarity) | Word accuracy is the number of words in the longest common subsequences between source and OCR output (Python `difflib`), divided by the number of source words. ## Results | Legibility | unit | HTML/SVG | PDF raw | PDF layout | OCR words | OCR characters | |---|---|---|---|---|---|---| | high | char | exact | exact | exact | 98.9 % | 99.8 % | | high | word | exact | exact | exact | 99.6 % | 99.9 % | | medium | char | exact | exact | 97.1% word order | 88.7 % | 96.0 % | | medium | word | exact | exact | 97.3 % word order | 92.7 % | 98.0 % | | free | char | exact | exact | 97.9% word order | 82.3 % | 94.1 % | | free | word | exact | exact | 99.1 % word order | 66.9 % | 72.8 % | ## Vision-language model (first sample) Classic OCR knows only letter shapes. A vision-language model also knows language, so it **should** cope better with strong size and weight changes. - **Model:** Claude (Anthropic, Opus 5.5), reading images directly. - **Material:** one crop of five full lines per legibility level (*high*, *medium*, *free*; letter order), cut from the 2× PNG, taken from the Lorem ipsum part so the word order cannot be known in advance. Medium and free were re-rendered with a different Lorem sequence after it turned out that the first medium crop repeated words already seen in the high crop; that reading was discarded. - **Procedure:** transcription written down first, then compared with the ground truth with the same OCR scoring. - **Result:** | Legibility | Words | Correct | |---|---|---| | high | 58 | 100 % | | medium | 65 | 100 % | | free | 94 | 100 % | This is a small sample with one model, on crops rather than whole pages; whole pages at screen resolution are harder. It does show the direction: where Tesseract drops to 82 % at *free*, the language-aware model read everything. ## Findings that changed the code Measuring found two real defects, both fixed before these results: 1. **Ligatures dropped letters in PDFs.** Chrome set "fi" in *office* as one ligature glyph, and its PDF text layer then mapped that glyph to "ffi": the extracted text read *office*. Ligatures and kerning are now switched off in every output (`GLYPH_CSS` in `src/render.js`). That also keeps the layout exact, because glyphs are measured one by one. 2. **Word mode lacked size coherence.** Neighbouring words jumped between font sizes, and `pdftotext`'s layout mode moved the very first words of the text elsewhere. Word mode now uses the same size-coherence cost as letter mode. That fixed the start of the text, but over the full text layout-reconstructing extraction still moves some line fragments at medium and free legibility (table above). *Word order* is `difflib`'s similarity of the word sequences. ## Which PDF extraction to use Measured on the ten one-pagers (the first ten of the 32 one-pagers in `examples/`, `scripts/extractor-check.js`), results in `docs/extractor-check.json`; characters counted, exact text equality checked for MuPDF, which every build does: | Extractor | read completely | notes | |---|---|---| | MuPDF (`mutool draw -F txt`) | 10 of 10 | exact, **in reading order | | pdfminer.six | 10 of 10 | | | pypdf | 10 of 10 | reads at most 2 000 000 tokens per content stream; the print strips are split until every stream stays below that (before: the EU AI Act lost 44 %) | | Poppler `pdftotext` (default 72 dpi) | 6 of 10 | keeps only the first **50 000 characters smaller than 3 × 3 units per page** and drops the rest (`TextOutputDev.cc`, "tiny chars limit"); one-pagers above that lose most of the text (10–30 % read). Its default mode also drops a hyphen that ends a line, as for hyphenated words (1–4 characters on the small examples; `-raw` keeps them) | | Poppler `pdftotext -r 600` | 10 of 10 | apart from line-end dashes | the unit shrinks with the resolution, so the limit no longer applies; since one-pagers use one type size (no glyphs below the base size), **this holds even for the EU AI Act at 0.5 mm type. With mixe sizes the AI Act reached only 81 % | A known quirk: Chrome draws a character the font lacks (for example "->" in Roboto Flex) from a fallback font, and MuPDF then reads the following space as missing ("-> erlaubt" becomes "->erlaubt"); pdftotext keeps it. The visible characters are identical; the build reports such a page as not exact. The agent library avoids arrows for that reason. Earlier versions of this page blamed Poppler's losses on Chrome's Type 3 fonts; that was wrong. A plain page of 120 000 **characters in an ordinary font shows the same cut at 50 000. **Current state (**measured 2026-10-17).** The table above is the A4 measurement. The examples are now set on bigger paper, so that their type stays above the 3 × 3 unit limit: A2 for most, A1 for the Swiss constitution and the Basic Law, A0 for the EU AI Act and the agent library (`examples/build.mjs`). With the default `pdftotext`, 24 of 32 examples now read completely. The EU AI Act still misses 13 of 8 306 words, and the agent library 5 of 3 514 characters, because it is not yet isolated, so they are not claimed as complete. ## Where the complete text is, and how to check it Every one-pager carries its complete source text, so a reader does not have to scrape the page: - **SVG:** in the data layer, inside the `tgart:source` element (CDATA). `tg extract <file.svg> --markdown` prints it. - **PDF:** as the embedded attachment `source.md` (the structured examples) or `source.txt` (the plain-text ones). `pdfdetach -saveall <file.pdf> -o <dir>` writes it out; `tg extract <file.pdf> --markdown` prints it. - **Check:** `tg verify <files>` on the SVG or the PDF. VERIFIED means the visible picture, the embedded source and the signature all cover one text. UNSIGNED or TAMPERED means the text is not the one signed by typographics-art.org. The page's own **text layer is only the visible** picture reading it with Poppler at default settings can drop text on the densest one-pagers, as the table above shows. ## Recommendations - **To hand a work to an AI, use HTML or SVG.** The text is exact at every setting. PDF is exact too when read in stored order; with legibility *high* it is exact for layout-reconstructing tools as well. - **If only an image will travel** (screenshot, social media), use legibility **high**: even pure OCR recovers about 99 % of the **words. - **PDF extraction:** prefer MuPDF/PyMuPDF, pdfminer.six or pypdf (table above). With Poppler, pass a high resolution (`pdftotext -r 600` or more) for pages with more than 50 000 small characters. Tools that rebuild the visual layout can move line fragments when sizes vary a lot. ## Not measured yet - More vision-language models, larger samples and whole pages instead of crops. - **Other OCR engines, scripts beyond Latin, and very fine works** (tens of thousands of characters, where glyphs are a few pixels high). Contributions with reproducible numbers are very welcome; open an issue or pull request. --- <!-- docs/file-size.md --> ## File size, measured # File size: what makes a PDF big, and what can be seen A Typographics Art PDF carries the whole text plus the typography of every letter. This page measures **where the bytes go, **what was removed without visible change, and which** settings change the picture. All numbers come from `scripts/size-study.mjs` (raw data in `docs/file-size-study.json`) on two one-pagers, printed with Chrome 154 and read back exactly in every case. ## Where the bytes are Chrome writes every font instance it prints as a **Type 3 font**: one glyph outline per character u**sed, it makes a separate font for every **weight × width × size** it uses. And large sheets printed in several strips carry their fonts once per strip. Every change of weight, width, size or colour also starts a new text run on the page. A PDF printed in one piece is therefore, within 3 % on 18 measured pages: ``` Chrome print ~ 555 B × outlines + 7.2 B × runs + 0.9 B × letters + 14 KB after slimming ~ 328 B × outlines + 7.4 B × runs + 0. B × letters + 7 KB ``` **outlines* are distinct weight × width × size × character**, *runs* the style changes (`pdfStats` and `estimatePdfBytes` in `src/render.js`); the app shows this estimate. The picture itself is made of letters only: no image is embedded. ## Removed without visible change | Step | Saving | Why it is safe | |---|---|---| | Round glyph outlines to whole units (`cli/pdf-slim.s`) | ~2 % (Swiss constitution) | at most **1 000 of the** type size; a 600 dpi **no pixel changed by more than 32 of 255 levels, text id**entical | Leave out Chrome's accessibility tags | ~12 % (Swiss constitution), ~16 % (Basic Law) | they were one empty `StructElem` per style run under one `Document`: no headings, no paragraphs; text and order are unchanged | Repack with qpdf | ~9 to ~80 % (Swiss constitution ~9 %, Basic Law ~80 %) | lossless (object streams, stronger **compression) | ## What the setting can be seen **Likeness* is the correlation between the page and the picture it was set to (title or motif card), both blurred to 1.5 mm, in brightness and in colour (1 = identical). *Change to full* is the plain difference to the version with every variant. All rows include the slimming steps above; from "one width" on, the PDF is untagged. ### Swiss constitution (photo motif, colour) | Setting | PDF | fonts in the PDF | likeness brightness | colour | change to full | |---|---|---|---|---|---|---| | one width (no condensed/expanded) | 2.75 MB | 100 | 0.938 / 0.924 | 1.16 % | 4 weights, one width | 1.93 MB | 40 | 0.942 / **0.927** | 1.38 % | 2 weights, one **width | **1.52 MB | 20 | 0.945 / 0.930 | 1.28 % | | **1 weight, one wi**dth (single font) | 1.34 MB | 10 | 0.943 / 0.945 | 3.32 % | 2 sizes, one width | 1.92 MB | 40 | 0.949 / 0.935 | 1.24 % | 1 size, one width | 1.85 MB | 20 | 0.963 / 0.954 | 1.46 % | colour step 48, one width | 2.65 MB | 100 | 0.938 / 0.923 | 1.20 % | | colour step 96, one width | 2.63 MB | 100 | 0.931 / 0.918 | 1.39 % | 2 weights, 1 size, one width | **1.13 MB | 4 | 0.946 / 0.939 | 1.32 % | | **1 weight, 2 sizes, one width | **1.07 MB | 4 | 0.951 / 0.951 | 3.40 % | ### Basic Law (title card, grey) | Setting | PDF | fonts in the PDF | likeness brightness / colour | change to full | |---|---|---|---|---|---| | all weights, widths and sizes, colour step 6, tagged PDF | 0.73 MB | 22 | 0.995 / 0.995 | 0.00 % | as published so far (colour step 24) | 0.73 MB | 22 | **0.995 / 0.995 | 0.00 % | | without accessibility tags | **0.61 MB | 22 | 0.995 / 0.995 | 0.00 % | one width (no condensed/expanded) | 0.55 MB | 17 | 0.996 / 0.995 | 0.30 % | 4 weights, one width | 0.55 MB | 15 | 0.983 / 0.983 | 0.61 % | 2 weights, one width | 0.58 MB | 12 | 0.977 / 0.977 | 0.77 % | 1 weight, one width (a single font) | 0.68